



AJOU UNIVERSITY



Numpy模块介绍

Gaoyang Shan

Mobile Multimedia Convergence Network Lab.

Contents

1 Nddarray对象, 数组属性, 数组创建

2 切片和索引

3 广播, 迭代数组, 数组操作, 与位运算

4 字符串, 数学, 算数, 统计, 及排序函数

5 字节交换, 副本和视图

6 矩阵库与线性代数

7 Numpy IO与Matplotlib

Ndarray对象与数组属性 (1/3)

- **Ndarray对象**是用于存放同类型元素的多维数组，内部由以下内容组成：
 - 一个指向数据(内存或内存映射文件中的一块数据)的指针。
 - 数据类型或 dtype，描述在数组中的固定大小值的格子。
 - 一个表示数组形状(shape)的元组，表示各维度大小的元组。
 - 一个跨度元组(stride)，其中的整数指的是为了前进到当前维度下一个元素需要"跨过"的字节数。

属性	说明
ndarray.ndim	秩，即轴的数量或维度的数量
ndarray.shape	数组的维度，对于矩阵，n 行 m 列
ndarray.size	数组元素的总个数，相当于 .shape 中 n*m 的值
ndarray.dtype	ndarray 对象的元素类型
ndarray.itemsize	ndarray 对象中每个元素的大小，以字节为单位
ndarray.flags	ndarray 对象的内存信息
ndarray.real	ndarray元素的实部
ndarray.imag	ndarray 元素的虚部
ndarray.data	包含实际数组元素的缓冲区，由于一般通过数组的索引获取元素，所以通常不需要使用这个属性。

Ndarray对象与数组属性 (2/3)

■ ndim

```
import numpy as np
a = np.arange(24)
print(a)
print(a.ndim)
b = a.reshape(2, 12)
print(b)
print(b.ndim)
```

■ 输出结果：

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]
1
[[ 0  1  2  3  4  5  6  7  8  9 10 11]
 [12 13 14 15 16 17 18 19 20 21 22 23]]
2
```

Ndarray对象与数组属性 (3/3)

■ shape

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape)
```

输出结果：

(2, 3)

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
a.shape = (3, 2)
print(a)
```

输出结果：

[[1 2]
[3 4]
[5 6]]

Ndarray数组创建 (1/4)

■ array()

```
import numpy as np
x = [1, 2, 3]
a = np.array(x)
print(a)
```

输出结果

[1 2 3]

■ asarray()

```
import numpy as np
x = [1, 2, 3]
a = np.asarray(x)
print(a)
```

输出结果

[1 2 3]

Ndarray数组创建 (2/4)

- arange()

```
import numpy as np
x = np.arange(5)
y = np.arange(5,dtype=float)
z = np.arange(10,20,2)
print(x)
print(y)
print(z)
```

输出结果：

[0 1 2 3 4]

[0. 1. 2. 3. 4.]

[10 12 14 16 18]

Ndarray数组创建 (3/4)

- linspace()

```
import numpy as np
a = np.linspace(1,10,10)
b = np.linspace(1,1,10)
print(a)
```

输出结果：

[1. 2. 3. 4. 5. 6. 7. 8. 9. 10.]

Ndarray数组创建 (4/4)

■ logspace()

```
import numpy as np
# 默认底数是 10
a = np.logspace(1.0, 2.0, num = 10)
#将底数设置为2
b = np.logspace(0,9,10,base=2)
print(a)
print(b)
```

输出结果：

```
[ 10.      12.91549665  16.68100537  21.5443469   27.82559402
 35.93813664  46.41588834  59.94842503  77.42636827 100.      ]
```

```
[ 1.  2.  4.  8. 16. 32. 64. 128. 256. 512.]
```

切片和索引(1/9)

- ndarray数组可以基于0-n的下标进行索引，切片对象可以通过内置的slice函数，并设置 start, stop, 及step参数进行，从原数组中切割出一个新数组。

```
import numpy as np
a = np.arange(10)
b = slice(2, 7, 2) # 从索引 2 开始到索引 7 停止，间隔为2
print(a)
print(a[b])
```

输出结果：

```
[0 1 2 3 4 5 6 7 8 9]
[2 4 6]
```

切片和索引(2/9)

- ndarray数组也可以通过冒号分隔切片参数 start:stop:step 来进行切片操作

```
import numpy as np
a = np.arange(10)
b = a[5]
c = a[2:7:2] # 从索引 2 开始到索引 7 停止 , 间隔为 2
d=a[3:]
print(a)
print(b)
print(c)
print(d)
```

输出结果 :

```
[0 1 2 3 4 5 6 7 8 9]
5
[2 4 6]
[3 4 5 6 7 8 9]
```

切片和索引(3/9)

- **多维数组**中，切片还可以包括省略号 **...**，来使**选择元组**的**长度与数组的维度**相同。如果在行位置使用省略号，它将返回包含行中元素的 **ndarray**。

```
import numpy as np
a = np.array([[1, 2, 3], [3, 4, 5], [4, 5, 6]])
print(a[..., 1]) # 第2列元素
print(a[1, ...]) # 第2行元素
print(a[..., 1:]) # 第2列及剩下的所有元素
```

输出结果：

```
[2 4 5]
[3 4 5]
[[2 3]
 [4 5]
 [5 6]]
```

切片和索引(4/9)

■ 整数数组索引

- 以下实例获取数组中 (0,0), (1,1) 和 (2,0) 位置处的元素。

```
import numpy as np
x = np.array([[1, 2], [3, 4], [5, 6]])
y = x[[0, 1, 2], [0, 1, 0]]
print(y)
```

输出结果：

[1 4 5]

- 以下实例获取了4X3数组中的四个角的元素。行索引是[0,0]和[3,3]，而列索引是[0,2]和[0,2]。

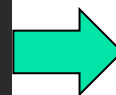
```
import numpy as np
x = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8], [9, 10, 11]])
print('我们的数组是：')
print(x)
rows = np.array([[0, 0], [3, 3]])
cols = np.array([[0, 2], [0, 2]])
y = x[rows, cols]
print('这个数组的四个角元素是：')
print(y)
```

我们的数组是：

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

这个数组的四个角元素是：

```
[[ 0  2]
 [ 9 11]]
```



切片和索引(5/9)

- 借助 : 或 ... 与索引切片组合

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
b = a[1:3, 1:3]
c = a[1:3, [1, 2]]
d = a[..., 1:]
print(b)
print(c)
print(d)
```

输出结果 :

```
[[5 6]
 [8 9]]
[[5 6]
 [8 9]]
[[2 3]
 [5 6]
 [8 9]]
```

切片和索引(6/9)

■ 布尔索引

- 布尔索引通过布尔运算（如：比较运算符）来获取符合指定条件的元素的数组。

```
import numpy as np
x = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8], [9, 10, 11]])
print('我们的数组是：')
print(y)
# 现在我们会打印出大于 5 的元素
print('大于 5 的元素是：')
print(x[x > 5])
```

输出结果：

我们的数组是：

[[0 8 9]

[7 4 5]

[6 7 8]

[9 10 11]]

大于 5 的元素是：

[6 7 8 9 10 11]

切片和索引(7/9)

■ 二维数组

■ 传入顺序索引数组

```
import numpy as np
x = np.arange(32).reshape((8, 4))
print(x)
# 二维数组读取指定下标对应的行
print("-----读取下标对应的行-----")
print(x[[4, 2, 1, 7]])
```

输出结果：

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]
 [28 29 30 31]]
-----读取下标对应的行-----
[[16 17 18 19]
 [ 8  9 10 11]
 [ 4  5  6  7]
 [28 29 30 31]]
```


切片和索引(8/9)

- 传入顺序索引数组

```
import numpy as np
x = np.arange(32).reshape((8, 4))
print(x[[-4, -2, -1, -7]])
```

输出结果：

```
[[16 17 18 19]
 [24 25 26 27]
 [28 29 30 31]
 [ 4  5  6  7]]
```

切片和索引(9/9)

- 代入多个索引数组

```
import numpy as np
x = np.arange(32).reshape((8, 4))
print(x)
print('\n',x[[1, 5, 7, 2], [0, 3, 1, 2]])
print('\n',x[np.ix_([1, 5, 7, 2], [0, 3, 1, 2])])
```

输出结果：

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]
 [24 25 26 27]
 [28 29 30 31]]
```

```
[ 4 23 29 10]
```

```
[[ 4  7  5  6]
 [20 23 21 22]
 [28 31 29 30]
 [ 8 11  9 10]]
```

广播与迭代数组(1/8)

- 广播(Broadcast)是 numpy 对不同形状(shape)的数组进行数值计算的方式, 对数组的算术运算通常在相应的元素上进行。
- 如果两个数组 a 和 b 形状相同, 即满足 `a.shape == b.shape`, 那么 `a*b` 的结果就是 a 与 b 数组对应位相乘。

```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
c = a * b
print(c)
```

- 输出结果 :

[10 40 90 160]

广播与迭代数组(2/8)

- 当运算中的 2 个数组的形状不同时, numpy 将自动触发广播机制。

```
import numpy as np
a = np.array([[0, 0, 0], [10, 10, 10], [20, 20, 20], [30, 30, 30]])
b = np.array([0, 1, 2])
print(a + b)
```

- 输出结果 :

```
[[ 0  1  2]
 [10 11 12]
 [20 21 22]
 [30 31 32]]
```

广播与迭代数组(3/8)

- NumPy迭代器对象`numpy.nditer`提供了一种灵活访问一个或者多个数组元素的方式。迭代器最基本的任务是可以完成对数组元素的访问。

```
import numpy as np
a = np.arange(1,7).reshape(2, 3)
print('原始数组是 : ')
print(a)
print('迭代输出元素 : ')
for x in np.nditer(a):
    print(x, end=" ")
print('\b\b')
```

- 输出结果 :

原始数组是 :

[[1 2 3]

[4 5 6]]

迭代输出元素 :

1, 2, 3, 4, 5, 6

广播与迭代数组(4/8)

- 迭代的顺序是和数组内存布局一致的，这样做是为了提升访问的效率，默认是行序优先。以下实例中，a和a.T的遍历顺序是一样的，因此他们在内存中的存储顺序也是一样的

```
import numpy as np
a = np.arange(6).reshape(2, 3)
b=a.T
print(b)
for x in np.nditer(b):
    print(x, end=", ")
print()
for x in np.nditer(b.copy(order='C')):
    print(x, end=", ")
print('\b\b')
```

- 输出结果：

```
[[0 3]
 [1 4]
 [2 5]]
0, 1, 2, 3, 4, 5,
0, 3, 1, 4, 2, 5
```

广播与迭代数组(5/8)

■ 控制遍历顺序

- For x in np.nditer(a, order='F'): Fortran order, 即是列序优先
- For x in np.nditer(a.T, order='C'): C order, 即是行序优先

```
import numpy as np
a = np.arange(0, 60, 5)
a = a.reshape(3, 4)
print('原始数组是 :')
print(a)
print('原始数组的转置是 :')
b = a.T
print(b)
print('以C风格顺序排序 :')
c = b.copy(order='C')
print(c)
for x in np.nditer(c):
    print(x, end=", ")
print('\b\b')
print('以 F 风格顺序排序 :')
c = b.copy(order='F')
print(c)
for x in np.nditer(c):
    print(x, end=", ")
```

输出结果

原始数组是 :

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

原始数组的转置是 :

```
[[ 0 20 40]
 [ 5 25 45]
 [10 30 50]
 [15 35 55]]
```

以 C 风格顺序排序 :

```
[[ 0 20 40]
 [ 5 25 45]
 [10 30 50]
 [15 35 55]]
```

0, 20, 40, 5, 25, 45, 10, 30, 50, 15, 35, 55

以 F 风格顺序排序 :

```
[[ 0 20 40]
 [ 5 25 45]
 [10 30 50]
 [15 35 55]]
```

0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55,
Process finished with exit code 0

广播与迭代数组(6/8)

- 可以通过显式设置, 来强制 `nditer` 对象使用某种顺序 :

```
import numpy as np
a = np.arange(0, 60, 5)
a = a.reshape(3, 4)
print('原始数组是 :')
print(a)
print('以 C 风格顺序排序 :')
for x in np.nditer(a, order='C'):
    print(x, end=", ")
print('\b\b')
print('以 F 风格顺序排序 :')
for x in np.nditer(a, order='F'):
    print(x, end=", ")
```



原始数组是 :

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

以 C 风格顺序排序 :

0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55

以 F 风格顺序排序 :

0, 20, 40, 5, 25, 45, 10, 30, 50, 15, 35, 55,

广播与迭代数组(7/8)

- **nditer**对象有另一个可选参数**op_flags**. 默认情况下, **nditer**将视待迭代遍历的数组为只读对象(read-only), 为了在遍历数组的同时, 实现对数组元素值得修改, 必须指定**readwrite**或者**writonly**的模式.

```
import numpy as np
a = np.arange(0,60,5)
a = a.reshape(3,4)
print('原始数组是 :')
print(a)
for x in np.nditer(a,
op_flags=['readwrite']):
    x[...] = 2*x
print('修改后的数组是 :')
print(a)
```



原始数组是 :

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

修改后的数组是 :

```
[[ 0 10 20 30]
 [40 50 60 70]
 [80 90 100 110]]
```

广播与迭代数组(8/8)

■ 广播迭代

- 如果两个数组是可广播的, nditer组合对象能够同时迭代它们. 假设数组a的维度为3X4, 数组 b 的维度为1X4, 则使用以下迭代器(数组 b 被广播到a的大小)。

```
import numpy as np
a = np.arange(0, 60, 5)
a = a.reshape(3, 4)
print('第一个数组为 :')
print(a)
print('第二个数组为 :')
b = np.array([1, 2, 3, 4], dtype=int)
print(b)
print('修改后的数组为 :')
for x, y in np.nditer([a, b]):
    print("%d:%d" % (x, y), end=", ")
```

输出结果

第一个数组为 :

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

第二个数组为 :

```
[1 2 3 4]
```

修改后的数组为 :

```
0:1, 5:2, 10:3, 15:4, 20:1, 25:2, 30:3, 35:4,
40:1, 45:2, 50:3, 55:4,
```

数组操作(1/25)

- Numpy 中包含了一些函数用于处理数组, 大概可分为以下几类:
 - 修改数组形状
 - 反转数组
 - 修改数组维度
 - 连接数组
 - 分割数组
 - 数组元素的添加与删除

数组操作(2/25)

- `numpy.reshape` 函数可以在不改变数据的条件下修改形状

```
import numpy as np
a = np.arange(8)
print('原始数组 :')
print(a)
b = a.reshape(4, 2)
print('修改后的数组 :')
print(b)
```



原始数组 :

[0 1 2 3 4 5 6 7]

修改后的数组 :

[[0 1]

[2 3]

[4 5]

[6 7]]

数组操作(5/25)

- `numpy.ndarray.flat`是一个数组元素迭代器, 实例如下:

```
import numpy as np
a = np.arange(9).reshape(3, 3)
print('原始数组 : ')
for row in a:
    print(row)
#
对数组中每个元素都进行处理
, 可以使用flat属性, 该属性是
一个数组元素迭代器 :
print('迭代后的数组 : ')
for element in a.flat:
    print(element)
```

输出结果

原始数组 :

[0 1 2]

[3 4 5]

[6 7 8]

迭代后的数组 :

0

1

2

3

4

5

6

7

8

数组操作(6/25)

- `numpy.ndarray.flatten` 返回一份数组拷贝, 对拷贝所做的修改不会影响原始数组

```
import numpy as np
a = np.arange(8).reshape(2, 4)
print('原数组 : ')
print(a)
# 默认按行
print('展开的数组 : ')
print(a.flatten())
print('以F风格顺序展开的数组 : ')
print(a.flatten(order='F'))
```

输出结果

原数组 :

[[0 1 2 3]

[4 5 6 7]]

展开的数组 :

[0 1 2 3 4 5 6 7]

以F风格顺序展开的数组 :

[0 4 1 5 2 6 3 7]

数组操作(7/25)

- `numpy.ravel()`展平的数组元素, 顺序通常是"C风格", 返回的是数组视图, 修改会影响原始数组。

```
import numpy as np
a = np.arange(8).reshape(2, 4)
print('原数组 : ')
print(a)
print('调用 ravel 函数之后 : ')
print(a.ravel())
print('以F风格顺序调用 ravel 函数之后 : ')
print(a.ravel(order='F'))
```

输出结果

原数组 :

```
[[0 1 2 3]
 [4 5 6 7]]
```

调用 ravel 函数之后 :

```
[0 1 2 3 4 5 6 7]
```

以F风格顺序调用 ravel 函数之后 :

```
[0 4 1 5 2 6 3 7]
```

数组操作(8/25)

■ numpy.transpose 函数用于对换数组的维度

```
import numpy as np
a = np.arange(12).reshape(3, 4)
print('原数组 : ')
print(a)
print('对换数组 : ')
print(np.transpose(a))
```

输出结果

原数组 :

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

对换数组 :

```
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

■ numpy.ndarray.T 类似numpy.transpose

```
import numpy as np
a = np.arange(12).reshape(3, 4)
print('原数组 : ')
print(a)
print('转置数组 : ')
print(a.T)
```

输出结果

数组操作(10/25)

■ numpy.swapaxes 函数用于交换数组的两个轴

```
import numpy as np
# 创建了三维的 ndarray
a = np.arange(8).reshape(2, 2, 2)
print('原数组 :')
print(a)
print('\n')
# 现在交换轴 0 (深度方向) 到轴
# 2 (宽度方向)
print('调用 swapaxes 函数后的数组 :')
print(np.swapaxes(a, 2, 0))
```



原数组 :

```
[[[0 1]
   [2 3]]
```

```
[[4 5]
 [6 7]]]
```

调用 swapaxes 函数后的数组 :

```
[[[0 4]
   [2 6]]
```

```
[[1 5]
 [3 7]]]
```

数组操作(11/25)

■ numpy.expand_dims 函数通过在指定位置插入新的轴来扩展数组形状

```
import numpy as np
x = np.array([[1, 2], [3, 4]])
print('数组 x : ')
print(x)
y = np.expand_dims(x, axis=0)
print('数组 y : ')
print(y)
print('数组 x 和 y 的形状 : ')
print(x.shape, y.shape)
# 在位置 1 插入轴
y = np.expand_dims(x, axis=1)
print('在位置 1 插入轴之后的数组 y : ')
print(y)
print('x.ndim 和 y.ndim : ')
print(x.ndim, y.ndim)
print('x.shape 和 y.shape : ')
print(x.shape, y.shape)
```



数组 x :

[[1 2]

[3 4]]

数组 y :

[[[1 2]

[3 4]]]

数组 x 和 y 的形状 :

(2, 2) (1, 2, 2)

在位置 1 插入轴之后的数组 y :

[[[1 2]

[3 4]]]

x.ndim 和 y.ndim :

2 3

x.shape 和 y.shape :

(2, 2) (2, 1, 2)

数组操作(12/25)

- `numpy.squeeze` 函数从给定数组的形状中删除一维的条目

```
import numpy as np
x = np.arange(9).reshape(3,1,3)
print('数组 x : ')
print(x)
y = np.squeeze(x)
print('数组 y : ')
print(y)
print('数组 x 和 y 的形状 : ')
print(x.shape, y.shape)
```



数组 x :
[[[0 1 2]]

[[3 4 5]]

[[6 7 8]]
数组 y :

[[0 1 2]

[3 4 5]

[6 7 8]]

数组 x 和 y 的形状 :
(3, 1, 3) (3, 3)

数组操作(13/25)

- **numpy.concatenate** 函数用于沿指定轴连接相同形状的两个或多个数组

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print('第一个数组 :')
print(a)
b = np.array([[5, 6], [7, 8]])
print('第二个数组 :')
print(b)
# 两个数组的维度相同
print('沿轴 0 连接两个数组 :')
print(np.concatenate((a, b)))
print('沿轴 1 连接两个数组 :')
print(np.concatenate((a, b), axis=1))
```



第一个数组 :

```
[[1 2]
 [3 4]]
```

第二个数组 :

```
[[5 6]
 [7 8]]
```

沿轴 0 连接两个数组 :

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

沿轴 1 连接两个数组 :

```
[[1 2 5 6]
 [3 4 7 8]]
```

数组操作(14/25)

■ numpy.stack 函数用于沿新轴连接数组序列

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print('第一个数组 :')
print(a)
b = np.array([[5, 6], [7, 8]])
print('第二个数组 :')
print(b)
print('沿轴 0 堆叠两个数组 :')
print(np.stack((a, b), 0))
print('沿轴 1 堆叠两个数组 :')
print(np.stack((a, b), 1))
```

输出结果

第一个数组 :

```
[[1 2]
```

```
[3 4]]
```

第二个数组 :

```
[[5 6]
```

```
[7 8]]
```

沿轴 0 堆叠两个数组 :

```
[[[1 2]
```

```
[3 4]
```

```
[[5 6]
```

```
[7 8]]]
```

沿轴 1 堆叠两个数组 :

```
[[[1 2]
```

```
[5 6]
```

```
[[3 4]
```

```
[7 8]]]
```

数组操作(15/25)

- `numpy.hstack` 是 `numpy.stack` 函数的变体, 它通过水平堆叠来生成数组。

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print('第一个数组 : ')
print(a)
b = np.array([[5, 6], [7, 8]])
print('第二个数组 : ')
print(b)
print('水平堆叠 : ')
c = np.hstack((a, b))
print(c)
```



第一个数组 :

[[1 2]

[3 4]]

第二个数组 :

[[5 6]

[7 8]]

水平堆叠 :

[[1 2 5 6]

[3 4 7 8]]

数组操作(16/25)

- `numpy.vstack` 是 `numpy.stack` 函数的变体, 它通过垂直堆叠来生成数组。

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print('第一个数组 :')
print(a)
b = np.array([[5, 6], [7, 8]])
print('第二个数组 :')
print(b)
print('竖直堆叠 :')
c = np.vstack((a, b))
print(c)
```



第一个数组 :

[[1 2]

[3 4]]

第二个数组 :

[[5 6]

[7 8]]

竖直堆叠 :

[[1 2]

[3 4]

[5 6]

[7 8]]

数组操作(17/25)

■ numpy.split 函数沿特定的轴将数组分割为子数组

```
import numpy as np
a = np.arange(9)
print('第一个数组 : ')
print(a)
print('将数组分为三个大小相等的子数组 : ')
b = np.split(a, 3)
print(b)
print('将数组在一维数组中表明的位置分割 : ')
b = np.split(a, [4, 7])
print(b)
```



第一个数组 :

[0 1 2 3 4 5 6 7 8]

将数组分为三个大小相等的子数组 :

[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]

将数组在一维数组中表明的位置分割 :

[array([0, 1, 2, 3]), array([4, 5, 6]), array([7, 8])]

数组操作(18/25)

- axis 为 0 时在水平方向分割, axis 为 1 时在垂直方向分割

```
import numpy as np
a = np.arange(16).reshape(4, 4)
print('第一个数组 :')
print(a)
print('默认分割 ( 0轴 ) :')
b = np.split(a,2)
print(b)
print('沿垂直方向分割 :')
c = np.split(a,2,1)
print(c)
print('沿垂直方向分割 :')
d = np.hsplit(a,2)
print(d)
```



第一个数组 :

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

默认分割 (0轴) :

```
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])]
```

沿垂直方向分割 :

```
[array([[ 0,  1],
        [ 4,  5],
        [ 8,  9],
        [12, 13]]), array([[ 2,  3],
        [ 6,  7],
        [10, 11],
        [14, 15]])]
```

沿垂直方向分割 :

```
[array([[ 0,  1],
        [ 4,  5],
        [ 8,  9],
        [12, 13]]), array([[ 2,  3],
        [ 6,  7],
        [10, 11],
        [14, 15]])]
```

数组操作(19/25)

- **numpy.hspllit** 函数用于水平分割数组, 通过指定要返回的相同形状的数量来拆分原数组。

```
import numpy as np
harr = np.floor(10 * np.random.random((2, 6)))
print('原array : ')
print(harr)
print('拆分后 : ')
print(np.hspllit(harr, 3))
```



原array :

```
[[4. 5. 2. 5. 5. 6.]
 [7. 4. 6. 4. 3. 7.]]
```

拆分后 :

```
[array([[4., 5.],
        [7., 4.]]), array([[2., 5.],
        [6., 4.]]), array([[5., 6.],
        [3., 7.]])]
```

数组操作(20/25)

- `numpy.vsplit` 沿着垂直轴分割, 其分割方式与`hsplit`用法相同。

```
import numpy as np
a = np.arange(16).reshape(4, 4)
print('第一个数组 :')
print(a)
print('水平分割 :')
b = np.vsplit(a, 2)
print(b)
```

输出结果

第一个数组 :

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

水平分割 :

```
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])]
```

数组操作(21/25)

- `numpy.resize` 函数返回指定大小的新数组。
- 如果新数组大小大于原始大小，则包含原始数组中的元素的副本。

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print('第一个数组：')
print(a)
print('第一个数组的形状：')
print(a.shape)
b = np.resize(a, (3, 2))
print('第二个数组：')
print(b)
print('第二个数组的形状：')
print(b.shape)
# 要注意 a 的第一行在 b
# 中重复出现，因为尺寸变大了
print('修改第二个数组的大小：')
b = np.resize(a, (3, 3))
print(b)
```

输出结果

第一个数组：

```
[[1 2 3]
 [4 5 6]]
```

第一个数组的形状：

```
(2, 3)
```

第二个数组：

```
[[1 2]
 [3 4]
 [5 6]]
```

第二个数组的形状：

```
(3, 2)
```

修改第二个数组的大小：

```
[[1 2 3]
 [4 5 6]
 [1 2 3]]
```

数组操作(22/25)

- `numpy.append` 函数在数组的末尾添加值。追加操作会分配整个数组，并把原来的数组复制到新数组中。此外，输入数组的维度必须匹配否则将生成 `ValueError`。
- `append` 函数返回的始终是一个一维数组。

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print('第一个数组 :')
print(a)
print('向数组添加元素 :')
print(np.append(a, [7, 8, 9]))
print('沿轴 0 添加元素 :')
print(np.append(a, [[7, 8, 9]], axis=0))
print('沿轴 1 添加元素 :')
print(np.append(a, [[5, 5, 5], [7, 8, 9]], axis=1))
```



第一个数组 :

[[1 2 3]

[4 5 6]]

向数组添加元素 :

[1 2 3 4 5 6 7 8 9]

沿轴 0 添加元素 :

[[1 2 3]

[4 5 6]

[7 8 9]]

沿轴 1 添加元素 :

[[1 2 3 5 5 5]

[4 5 6 7 8 9]]

数组操作(23/25)

- `numpy.insert` 函数在给定索引之前，沿给定轴在输入数组中插入值。
- 如果值的类型转换为要插入，则它与输入数组不同。插入没有原地的，函数会返回一个新数组。此外，如果未提供轴，则输入数组会被展开。

```
import numpy as np
a = np.array([[1, 2], [3, 4], [5, 6]])
print('第一个数组 :')
print(a)
print('未传递 Axis 参数。 在删除之前输入数组会被展开。')
print(np.insert(a, 3, [11, 12]))
print('传递了 Axis 参数。 会广播值数组来配输入数组。')
print('沿轴 0 广播 :')
print(np.insert(a, 1, [11], axis=0))
print('沿轴 1 广播 :')
print(np.insert(a, 1, 11, axis=1))
```

输出结果

第一个数组 :

```
[[1 2]
 [3 4]
 [5 6]]
```

未传递 Axis 参数。 在删除之前输入数组会被展开。

```
[ 1  2  3 11 12  4  5  6]
```

传递了 Axis 参数。 会广播值数组来配输入数组。

沿轴 0 广播 :

```
[[ 1  2]
 [11 11]
 [ 3  4]
 [ 5  6]]
```

沿轴 1 广播 :

```
[[ 1 11  2]
 [ 3 11  4]
 [ 5 11  6]]
```

数组操作(24/25)

- **numpy.delete** 函数返回从输入数组中删除指定子数组的新数组。与 **insert()** 函数的情况一样，如果未提供轴参数，则输入数组将展开。

```
import numpy as np
a = np.arange(12).reshape(3, 4)
print('第一个数组 :')
print(a)
print('未传递 Axis 参数。
在插入之前输入数组会被展开。')
print(np.delete(a, 5))
print('删除第二列 :')
print(np.delete(a, 1, axis=1))
print('包含从数组中删除的替代值的切片 :')
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(np.delete(a, np.s_[:2]))
```

输出结果

第一个数组 :

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

未传递 Axis 参数。在插入之前输入数组会被展开。

```
[ 0  1  2  3  4  6  7  8  9 10 11]
```

删除第二列 :

```
[[ 0  2  3]
 [ 4  6  7]
 [ 8 10 11]]
```

包含从数组中删除的替代值的切片 :

```
[ 2  4  6  8 10]
```

数组操作(25/25)

- **numpy.unique** 函数用于去除数组中的重复元素。

```
import numpy as np
a = np.array([5, 2, 6, 2, 7, 5, 6, 8, 2, 9])
print('第一个数组 : ')
print(a)
print('第一个数组的去重值 : ')
u = np.unique(a)
print(u)
print('去重数组的索引数组 : ')
u, indices = np.unique(a, return_index=True)
print(indices)
print('我们可以看到每个和原数组下标对应的数值 : ')
print(a)
print('去重数组的下标 : ')
u, indices = np.unique(a, return_inverse=True)
print(u)
print('下标为 : ')
print(indices)
print('使用下标重构原数组 : ')
print(u[indices])
print('返回去重元素的重复数量 : ')
u, indices = np.unique(a, return_counts=True)
print(u)
print(indices)
```

输出结果

第一个数组 :

[5 2 6 2 7 5 6 8 2 9]

第一个数组的去重值 :

[2 5 6 7 8 9]

去重数组的索引数组 :

[1 0 2 4 7 9]

我们可以看到每个和原数组下标对应的数值 :

[5 2 6 2 7 5 6 8 2 9]

去重数组 :

[2 5 6 7 8 9]

原数组在去重数组的下标为 :

[1 0 2 0 3 1 2 4 0 5]

使用下标重构原数组 :

[5 2 6 2 7 5 6 8 2 9]

返回去重元素的重复数量 :

[2 5 6 7 8 9]

[3 2 2 1 1 1]

位运算(1/7)

- Numpy “bitwise_”开头的函数是位运算函数. Numpy位运算包括以下几个函数

函数	描述
bitwise_and	对数组元素执行位与操作
bitwise_or	对数组元素执行位或操作
invert	按位取反
left_shift	向左移动二进制表示的位
right_shift	向右移动二进制表示的位

位运算(2/7)

- `bitwise_and()` 函数对数组中整数的二进制形式执行位与运算。

```
import numpy as np
print('13 和 17 的二进制形式 :')
a, b = 13, 17
print(bin(a), bin(b))
print('13 和 17 的位与 :')
print(np.bitwise_and(13, 17))
```

输出结果

13 和 17 的二进制形式 :
0b1101 0b10001
13 和 17 的位与 :
1

位运算(3/7)

- `bitwise_or()`函数对数组中整数的二进制形式执行位或运算。

```
import numpy as np
a, b = 13, 17
print('13 和 17 的二进制形式 :')
print(bin(a), bin(b))
print('13 和 17 的位或 :')
print(np.bitwise_or(13, 17))
```



13 和 17 的二进制形式 :
0b1101 0b10001
13 和 17 的位或 :
29

位运算(4/7)

- `invert()`函数对数组中整数进行位取反运算，即 0 变成 1，1 变成 0。

```
import numpy as np
print('13的位反转，其中ndarray的dtype是uint8 :')
print(np.invert(np.array([13], dtype=np.uint8)))
print('13的二进制表示:')
print(np.binary_repr(13, width=8))
print('242的二进制表示 :')
print(np.binary_repr(242, width=8))
```



13的位反转，其中ndarray的dtype是uint8 :
[242]

13的二进制表示：

00001101

242的二进制表示：

11110010

位运算(6/7)

- `invert()`函数对数组中整数进行位取反运算，即 0 变成 1，1 变成 0。

```
import numpy as np
print('将 10 左移两位 :')
print(np.left_shift(10, 2))
print('10 的二进制表示 :')
print(np.binary_repr(10, width=8))
print('40 的二进制表示 :')
print(np.binary_repr(40, width=8))
```



将 10 左移两位 :

40

10 的二进制表示 :

00001010

40 的二进制表示 :

00101000

位运算(7/7)

- `right_shift()`函数将数组元素的二进制形式向右移动到指定位置，左侧附加相等数量的0。

```
import numpy as np
print('将 40 右移两位 :')
print(np.right_shift(40, 2))
print('40 的二进制表示 :')
print(np.binary_repr(40, width=8))
print('10 的二进制表示 :')
print(np.binary_repr(10, width=8))
# '00001010'
中的两位移动到了右边，并在左边
添加了两个 0。
```



将 40 右移两位 :
10
40 的二进制表示 :
00101000
10 的二进制表示 :
00001010

字符串函数(1/8)

- 以下函数用于对dtype为numpy.string_或numpy.unicode_的数组执行向量化字符串操作。它们基于Python内置库中的标准字符串函数。
- 这些函数在字符数组类(numpy.char)中定义。

函数	描述
add()	对两个数组的逐个字符串元素进行连接
multiply()	返回按元素多重连接后的字符串
center()	居中字符串
capitalize()	将字符串第一个字母转换为大写
title()	将字符串的每个单词的第一个字母转换为大写
lower()	数组元素转换为小写
upper()	数组元素转换为大写
split()	指定分隔符对字符串进行分割, 并返回数组列表
splitlines()	返回元素中的行列表, 以换行符分割
strip()	移除元素开头或者结尾处的特定字符
join()	通过指定分隔符来连接数组中的元素
replace()	使用新字符串替换字符串中的所有子字符串
decode()	数组元素依次调用str.decode
encode()	数组元素依次调用str.encode

字符串函数(2/8)

- `numpy.char.add()` 函数依次对两个数组的元素进行字符串连接。

```
import numpy as np
print('连接两个字符串 : ')
print(np.char.add(['hello'], [' xyz']))
print('连接示例 : ')
print(np.char.add(['hello', 'hi'], [' abc', ' xyz']))
```



连接两个字符串 :
['hello xyz']
连接示例 :
['hello abc' 'hi xyz']

- `numpy.char.multiply()` 函数执行多重连接。

```
import numpy as np
print(np.char.multiply('Runoob ', 3))
```



Runoob Runoob Runoob

字符串函数(3/8)

- `numpy.char.center()` 函数用于将字符串居中，并使用指定字符在左侧和右侧进行填充。

```
import numpy as np
# np.char.center(str, width, fillchar) :
# str: 字符串, width: 长度, fillchar: 填充字符
print(np.char.center('Runoob', 20, fillchar='*'))
```



*****Runoob*****

字符串函数(4/8)

- **numpy.char.capitalize()**函数将字符串的第一个字母转换为大写：

```
import numpy as np
print(np.char.capitalize('runoob'))
```

输出结果

Runoob

- **numpy.char.title()**函数将字符串的每个单词的第一个字母转换为大写：

```
import numpy as np
print(np.char.title('i like runoob'))
```

输出结果

I Like Runoob

- **numpy.char.lower()**函数对数组的每个元素转换为小写。它对每个元素调用 **str.lower**。

```
import numpy as np
# 操作数组
print(np.char.lower(['RUNOOB', 'GOOGLE']))
# 操作字符串
print(np.char.lower('RUNOOB'))
```

输出结果

['runoob' 'google']
runoob

字符串函数(5/8)

- `numpy.char.upper()` 函数对数组的每个元素转换为大写。它对每个元素调用 `str.upper`。

```
import numpy as np
# 操作数组
print(np.char.upper(['runoob', 'google']))
# 操作字符串
print(np.char.upper('runoob'))
```

输出结果 → ['RUNOOB' 'GOOGLE']
RUNOOB

- `numpy.char.split()` 通过指定分隔符对字符串进行分割，并返回数组。默认情况下，分隔符为空格。

```
import numpy as np
# 分隔符默认为空格
print(np.char.split('i like runoob?'))
# 分隔符为 .
print(np.char.split('www.runoob.com', sep='.'))
```

输出结果 → ['i', 'like', 'runoob?']
['www', 'runoob', 'com']

字符串函数(6/8)

- `numpy.char.splitlines()` 函数以换行符作为分隔符来分割字符串，并返回数组。

```
import numpy as np
# 换行符 \n
print(np.char.splitlines('i\nlike runoob?'))
print(np.char.splitlines('i\rlike runoob?'))
```



```
['i', 'like runoob?']
['i', 'like runoob?']
```

- `numpy.char.strip()` 函数用于移除开头或结尾处的特定字符。

```
import numpy as np
# 移除字符串头尾的 a 字符
print(np.char.strip('ashok arunooba', 'a'))
# 移除数组元素头尾的 a 字符
print(np.char.strip(['arunooba', 'admin', 'java'], 'a'))
```



```
shok arunoob
['runoob' 'dmin' 'jav']
```

字符串函数(7/8)

- **numpy.char.join()** 函数通过指定分隔符来连接数组中的元素或字符串

```
import numpy as np
# 操作字符串
print(np.char.join(':', 'runoob'))
# 指定多个分隔符操作数组元素
print(np.char.join([':', '-'], ['runoob', 'google']))
```



r:u:n:o:o:b
[r:u:n:o:o:b 'g-o-o-g-l-e']

- **numpy.char.replace()** 函数使用新字符串替换字符串中的所有子字符串。

```
import numpy as np
print(np.char.replace('i like runoob', 'oo', 'cc'))
```



i like runcb

字符串函数(8/8)

- `numpy.char.encode()` 函数对数组中的每个元素调用 `str.encode` 函数。默认编码是 `utf-8`, 可以使用标准 Python 库中的编解码器。
- `numpy.char.decode()` 函数对编码的元素进行 `str.decode()` 解码。

```
import numpy as np
a = np.char.encode('runoob', 'cp500')
print(a)
print(np.char.decode(a, 'cp500'))
```



```
b'\x99\xa4\x95\x96\x96\x82'
runoob
```

数学, 算数, 统计函数(1/15)

■ 三角函数

- NumPy 提供了标准的三角函数：sin()、cos()、tan()。

```
import numpy as np
a = np.array([0, 30, 45, 60, 90])
print('不同角度的正弦值：')
# 通过乘 pi/180 转化为弧度
print(np.sin(a * np.pi / 180))
print('数组中角度的余弦值：')
print(np.cos(a * np.pi / 180))
print('数组中角度的正切值：')
print(np.tan(a * np.pi / 180))
```

输出结果

不同角度的正弦值：

[0. 0.5 0.70710678 0.8660254
1.]

数组中角度的余弦值：

[1.00000000e+00 8.66025404e-01
7.07106781e-01 5.00000000e-01
6.12323400e-17]

数组中角度的正切值：

[0.00000000e+00 5.77350269e-01
1.00000000e+00 1.73205081e+00
1.63312394e+16]

数学, 算数, 统计函数(2/15)

- **arcsin, arccos, 和 arctan 函数返回给定角度的 sin, cos 和 tan 的反三角函数。这些函数的结果可以通过 numpy.degrees() 函数将弧度转换为角度。**

```
import numpy as np
a = np.array([0, 30, 45, 60, 90])
print('含有正弦值的数组 : ')
sin = np.sin(a * np.pi / 180)
print(sin)
print('计算角度的反正弦, 返回值以弧度为单位 : ')
inv = np.arcsin(sin)
print(inv)
print('通过转化为角度制来检查结果 : ')
print(np.degrees(inv))
print('arccos 和 arctan 函数行为类似 : ')
cos = np.cos(a * np.pi / 180)
print(cos)
print('反余弦 : ')
inv = np.arccos(cos)
print(inv)
print('角度制单位 : ')
print(np.degrees(inv))
print('tan 函数 : ')
tan = np.tan(a * np.pi / 180)
print(tan)
print('反正切 : ')
inv = np.arctan(tan)
print(inv)
print('角度制单位 : ')
print(np.degrees(inv))
```



含有正弦值的数组 :

[0. 0.5 0.70710678 0.8660254 1.]

计算角度的反正弦, 返回值以弧度为单位 :

[0. 0.52359878 0.78539816 1.04719755 1.57079633]

通过转化为角度制来检查结果 :

[0. 30. 45. 60. 90.]

arccos 和 arctan 函数行为类似 :

[1.00000000e+00 8.66025404e-01 7.07106781e-01

5.00000000e-01

6.12323400e-17]

反余弦 :

[0. 0.52359878 0.78539816 1.04719755 1.57079633]

角度制单位 :

[0. 30. 45. 60. 90.]

tan 函数 :

[0.00000000e+00 5.77350269e-01 1.00000000e+00

1.73205081e+00

1.63312394e+16]

反正切 :

[0. 0.52359878 0.78539816 1.04719755 1.57079633]

角度制单位 :

[0. 30. 45. 60. 90.]

数学, 算数, 统计函数(3/15)

- `numpy.around()` 函数返回指定数字的四舍五入值。

```
import numpy as np
a = np.array([1.0, 5.55, 123, 0.567, 25.532])
print('原数组 : ')
print(a)
print('舍入后 : ')
print(np.around(a))
print(np.around(a, decimals=1))
print(np.around(a, decimals=-1))
```

输出结果

原数组 :

[1. 5.55 123. 0.567 25.532]

舍入后 :

[1. 6. 123. 1. 26.]

[1. 5.6 123. 0.6 25.5]

[0. 10. 120. 0. 30.]

数学, 算数, 统计函数(4/15)

- **numpy.floor()**返回小于或者等于指定表达式的最大整数, 即向下取整。

```
import numpy as np
a = np.array([-1.7, 1.5, -0.2, 0.6, 10])
print('提供的数组 : ')
print(a)
print('修改后的数组 : ')
print(np.floor(a))
```



提供的数组 :

[-1.7 1.5 -0.2 0.6 10.]

修改后的数组 :

[-2. 1. -1. 0. 10.]

- **numpy.ceil()** 返回大于或者等于指定表达式的最小整数, 即向上取整。

```
import numpy as np
a = np.array([-1.7, 1.5, -0.2, 0.6, 10])
print('提供的数组 : ')
print(a)
print('修改后的数组 : ')
print(np.ceil(a))
```



提供的数组 :

[-1.7 1.5 -0.2 0.6 10.]

修改后的数组 :

[-1. 2. -0. 1. 10.]

数学, 算数, 统计函数(5/15)

- NumPy算术函数包含简单的加减乘除: `add()`, `subtract()`, `multiply()` 和 `divide()`。
 - 需要注意的是数组必须具有相同的形状或符合数组广播规则。

```
import numpy as np
a = np.arange(9, dtype=np.float_).reshape(3, 3)
print('第一个数组 :')
print(a)
print('第二个数组 :')
b = np.array([10, 10, 10])
print(b)
print('两个数组相加 :')
print(np.add(a, b))
print('两个数组相减 :')
print(np.subtract(a, b))
print('两个数组相乘 :')
print(np.multiply(a, b))
print('两个数组相除 :')
print(np.divide(a, b))
```

输出结果

第一个数组 :
[[0. 1. 2.]
 [3. 4. 5.]
 [6. 7. 8.]
第二个数组 :
[10 10 10]
两个数组相加 :
[[10. 11. 12.]
 [13. 14. 15.]
 [16. 17. 18.]
两个数组相减 :
[[-10. -9. -8.]
 [-7. -6. -5.]
 [-4. -3. -2.]
两个数组相乘 :
[[0. 10. 20.]
 [30. 40. 50.]
 [60. 70. 80.]
两个数组相除 :
[[0. 0.1 0.2]
 [0.3 0.4 0.5]
 [0.6 0.7 0.8]]

数学, 算数, 统计函数(6/15)

- `numpy.reciprocal()` 函数返回参数逐元素的倒数。如 $1/4$ 倒数为 $4/1$ 。

```
import numpy as np
a = np.array([0.25, 1.33, 1, 100])
print('我们的数组是 :')
print(a)
print('调用 reciprocal 函数 :')
print(np.reciprocal(a))
```

输出结果

我们的数组是 :

[0.25 1.33 1. 100.]

调用 reciprocal 函数 :

[4. 0.7518797 1. 0.01]

- `numpy.power()` 函数将第一个输入数组中的元素作为底数, 计算它与第二个输入数组中相应元素的幂。

```
import numpy as np
a = np.array([10, 100, 1000])
print('我们的数组是 ;')
print(a)
print('调用 power 函数 :')
print(np.power(a, 2))
print('第二个数组 :')
b = np.array([1, 2, 3])
print(b)
print('再次调用 power 函数 :')
print(np.power(a, b))
```

输出结果

我们的数组是 ;

[10 100 1000]

调用 power 函数 :

[100 10000 1000000]

第二个数组 :

[1 2 3]

再次调用 power 函数 :

[10 10000 10000000000]

数学, 算数, 统计函数(7/15)

- `numpy.mod()` 计算输入数组中相应元素的相除后的余数。函数 `numpy.remainder()` 也产生相同的结果。

```
import numpy as np
a = np.array([10, 20, 30])
b = np.array([3, 5, 7])
print('第一个数组 :')
print(a)
print('第二个数组 :')
print(b)
print('调用 mod() 函数 :')
print(np.mod(a, b))
print('调用 remainder() 函数 :')
print(np.remainder(a, b))
```

输出结果

第一个数组 :

[10 20 30]

第二个数组 :

[3 5 7]

调用 `mod()` 函数 :

[1 0 2]

调用 `remainder()` 函数 :

[1 0 2]

数学, 算数, 统计函数(8/15)

- `numpy.amin()` 用于计算数组中的元素沿指定轴的最小值。
- `numpy.amax()` 用于计算数组中的元素沿指定轴的最大值。

```
import numpy as np
a = np.array([[3, 7, 5], [8, 4, 3], [2, 4, 9]])
print('我们的数组是 :')
print(a)
print('调用 amin() 函数 :')
print(np.amin(a))
print('再次调用 amin() 函数 :')
print(np.amin(a, 1))
print('再次调用 amin() 函数 :')
print(np.amin(a, 0))
print('调用 amax() 函数 :')
print(np.amax(a))
print('再次调用 amax() 函数 :')
print(np.amax(a, 0))
print('再次调用 amax() 函数 :')
print(np.amax(a, 1))
```

输出结果

我们的数组是 :

`[[3 7 5]`

`[8 4 3]`

`[2 4 9]]`

调用 `amin()` 函数 :

`2`

再次调用 `amin()` 函数 :

`[3 3 2]`

再次调用 `amin()` 函数 :

`[2 4 3]`

调用 `amax()` 函数 :

`9`

再次调用 `amax()` 函数 :

`[8 7 9]`

再次调用 `amax()` 函数 :

`[7 8 9]`

数学, 算数, 统计函数(9/15)

- `numpy.ptp()`函数计算数组中元素最大值与最小值的差(最大值 - 最小值)。

```
import numpy as np
a = np.array([[3, 7, 5], [8, 4, 3], [2, 4, 9]])
print('我们的数组是 :')
print(a)
print('调用 ptp() 函数 :')
print(np.ptp(a))
print('沿轴 1 调用 ptp() 函数 :')
print(np.ptp(a, 1))
print('沿轴 0 调用 ptp() 函数 :')
print(np.ptp(a, 0))
```



我们的数组是 :

[[3 7 5]

[8 4 3]

[2 4 9]]

调用 `ptp()` 函数 :

7

沿轴 1 调用 `ptp()` 函数 :

[4 5 7]

沿轴 0 调用 `ptp()` 函数 :

[6 3 6]

数学, 算数, 统计函数(10/15)

■ numpy.percentile()

- 百分位数是统计中使用的度量, 表示小于这个值的观察值的百分比

```
import numpy as np
a = np.array([[10, 7, 4], [3, 2, 1]])
print('我们的数组是 :')
print(a)
print('调用percentile()函数 :')
# 50% 的分位数, 就是 a 里排序之后的中位数
print(np.percentile(a, 50))
# axis 为 0, 在纵列上求
print(np.percentile(a, 50, axis=0))
# axis 为 1, 在横行上求
print(np.percentile(a, 50, axis=1))
# 保持维度不变
print (np.percentile(a, 50, axis=1, keepdims=True))
```

输出结果

我们的数组是 :

[[10 7 4]

[3 2 1]]

调用percentile()函数 :

3.5

[6.5 4.5 2.5]

[7. 2.]

[[7.]

[2.]]

数学, 算数, 统计函数(11/15)

■ numpy.median()函数用于计算数组a中元素的中位数(中值)

```
import numpy as np
a = np.array([[30, 65, 70], [80, 95, 10], [50, 90, 60]])
print('我们的数组是 : ')
print(a)
print('调用 median() 函数 : ')
print(np.median(a))
print('沿轴 0 调用 median() 函数 : ')
print(np.median(a, axis=0))
print('沿轴 1 调用 median() 函数 : ')
print(np.median(a, axis=1))
```

输出结果

我们的数组是 :

[[30 65 70]

[80 95 10]

[50 90 60]]

调用 median() 函数 :

65.0

沿轴 0 调用 median() 函数 :

[50. 90. 60.]

沿轴 1 调用 median() 函数 :

[65. 80. 60.]

数学, 算数, 统计函数(12/15)

- `numpy.mean()` 函数返回数组中元素的算术平均值。如果提供了轴, 则沿其计算。

```
import numpy as np
a = np.array([[1, 2, 3], [3, 4, 5], [4, 5, 6]])
print('我们的数组是 : ')
print(a)
print('调用 mean() 函数 : ')
print(np.mean(a))
print('沿轴 0 调用 mean() 函数 : ')
print(np.mean(a, axis=0))
print('沿轴 1 调用 mean() 函数 : ')
print(np.mean(a, axis=1))
```

输出结果

我们的数组是 :

[[1 2 3]

[3 4 5]

[4 5 6]]

调用 mean() 函数 :

3.6666666666666665

沿轴 0 调用 mean() 函数 :

[2.66666667 3.66666667 4.66666667]

沿轴 1 调用 mean() 函数 :

[2. 4. 5.]

数学, 算数, 统计函数(13/15)

- `numpy.average()` 函数根据在另一个数组中给出的各自的权重计算数组中元素的加权平均值。
- 该函数可以接受一个轴参数。如果没有指定轴, 则数组会被展开。
- 加权平均值即将各数值乘以相应的权数, 然后加总求和得到总体值, 再除以总的单位数。
- 考虑数组`[1,2,3,4]`和相应的权重`[4,3,2,1]`, 通过将相应元素的乘积相加, 并将和除以权重的和, 来计算加权平均值。

```
import numpy as np
a = np.array([1, 2, 3, 4])
print('我们的数组是 :')
print(a)
print('调用 average() 函数 :')
print(np.average(a))
# 不指定权重时相当于 mean 函数
wts = np.array([4, 3, 2, 1])
print('再次调用 average() 函数 :')
print(np.average(a, weights=wts))
# 如果 returned 参数设为 true , 则返回权重的和
print('权重的和 :')
print(np.average([1, 2, 3, 4], weights=[4, 3, 2, 1], returned=True))
```

输出结果

我们的数组是 :

`[1 2 3 4]`

调用 `average()` 函数 :

`2.5`

再次调用 `average()` 函数 :

`2.0`

权重的和 :

`(2.0, 10.0)`

数学, 算数, 统计函数(14/15)

- `numpy.average`在多维数组中, 可以指定用于计算的轴。

```
import numpy as np
a = np.arange(6).reshape(3, 2)
print('我们的数组是 :')
print(a)
print('修改后的数组 :')
wt = np.array([3, 5])
print(np.average(a, axis=1, weights=wt))
print('修改后的数组 :')
print(np.average(a, axis=1, weights=wt, returned=True))
```

输出结果

我们的数组是 :

```
[[0 1]
 [2 3]
 [4 5]]
```

修改后的数组 :

```
[0.625 2.625 4.625]
```

修改后的数组 :

```
(array([0.625, 2.625, 4.625]), array([8., 8., 8.]))
```

数学, 算数, 统计函数(15/15)

■ numpy.std()

- 标准差是一组数据平均值分散程度的一种度量。
- 标准差是方差的算术平方根。

```
import numpy as np  
print(np.std([1, 2, 3, 4]))
```



1.118033988749895

■ numpy.var()

- 统计中的方差（样本方差）是每个样本值与全体样本值的平均数之差的平方值的平均数。
- 换句话说，标准差是方差的平方根。

```
import numpy as np  
print(np.var([1, 2, 3, 4]))
```



1.25

Numpy排序, 条件刷选函数(1/8)

- `numpy.sort()`函数返回输入数组的排序副本。

```
import numpy as np
a = np.array([[3, 7], [9, 1]])
print('我们的数组是 :')
print(a)
print('调用 sort() 函数 :')
print(np.sort(a))
print('按列排序 :')
print(np.sort(a, axis=0))
# 在 sort 函数中排序字段
dt = np.dtype([('name', 'S10'), ('age', int)])
a = np.array([('raju', 21), ('anil', 25), ('ravi', 17), ('amar', 27)], dtype=dt)
print('我们的数组是 :')
print(a)
print('按 name 排序 :')
print(np.sort(a, order='name'))
```

输出结果

我们的数组是 :

[[3 7]

[9 1]]

调用 sort() 函数 :

[[3 7]

[1 9]]

按列排序 :

[[3 1]

[9 7]]

我们的数组是 :

[(b'raju', 21) (b'anil', 25) (b'ravi', 17) (b'amar', 27)]

按 name 排序 :

[(b'amar', 27) (b'anil', 25) (b'raju', 21) (b'ravi', 17)]

Numpy排序, 条件刷选函数(2/8)

- `numpy.argsort()` 函数返回的是数组值从小到大的索引值。

```
import numpy as np
x = np.array([3, 1, 2])
print('我们的数组是 :')
print(x)
print('对 x 调用 argsort() 函数 :')
y = np.argsort(x)
print(y)
print('以排序后的顺序重构原数组 :')
print(x[y])
print('使用循环重构原数组 :')
for i in y:
    print(x[i], end=" ")
```



我们的数组是 :

[3 1 2]

对 x 调用 argsort() 函数 :

[1 2 0]

以排序后的顺序重构原数组 :

[1 2 3]

使用循环重构原数组 :

1 2 3

Numpy排序, 条件刷选函数(3/8)

- `numpy.lexsort()`用于对多个序列进行排序。把它想象成对电子表格进行排序, 每一列代表一个序列, 排序时优先照顾靠后的列。

```
import numpy as np
nm = ('raju', 'anil', 'ravi', 'amar')
dv = ('f.y.', 's.y.', 's.y.', 'f.y.')
ind = np.lexsort((dv, nm))
print('调用 lexsort() 函数 :')
print(ind)
print('使用这个索引来获取排序后的数据 :')
print([nm[i] + ", " + dv[i] for i in ind])
```

输出结果

调用 `lexsort()` 函数 :

[3 1 0 2]

使用这个索引来获取排序后的数据 :

['amar, f.y.', 'anil, s.y.', 'raju, f.y.', 'ravi, s.y.']

Numpy排序, 条件刷选函数(4/8)

■ 复数排序 : `numpy.sort_complex()`

```
import numpy as np
print(np.sort_complex([5, 3, 6, 2, 1]))
print(np.sort_complex([1 + 2j, 2 - 1j, 3 - 2j, 3 - 3j, 3 + 5j]))
```

输出结果

```
[1.+0.j 2.+0.j 3.+0.j 5.+0.j 6.+0.j]
[1.+2.j 2.-1.j 3.-3.j 3.-2.j 3.+5.j]
```

■ 分区排序 : `numpy.partition()`

```
import numpy as np
a = np.array([3, 4, 2, 1])
print(np.partition(a, 3))
print(np.partition(a, (1, 3)))
```

输出结果

```
[2 1 3 4]
[1 2 3 4]
```

Numpy排序, 条件刷选函数(5/8)

- `numpy.argmax()` 和 `numpy.argmin()` 函数分别沿给定轴返回最大和最小元素的索引。

```
import numpy as np
a = np.array([[30, 40, 70], [80, 20, 10], [50, 90, 60]])
print('我们的数组是 :')
print(a)
print('调用 argmax() 函数 :')
print(np.argmax(a))
print('展开数组 :')
print(a.flatten())
print('沿轴 0 的最大值索引 :')
maxindex = np.argmax(a, axis=0)
print(maxindex)
print('沿轴 1 的最大值索引 :')
maxindex = np.argmax(a, axis=1)
print(maxindex)
print('调用 argmin() 函数 :')
minindex = np.argmin(a)
print(minindex)
print('展开数组中的最小值 :')
print(a.flatten()[minindex])
print('沿轴 0 的最小值索引 :')
minindex = np.argmin(a, axis=0)
print(minindex)
print('沿轴 1 的最小值索引 :')
minindex = np.argmin(a, axis=1)
print(minindex)
```

输出结果

我们的数组是 :

`[[30 40 70]`

`[80 20 10]`

`[50 90 60]]`

调用 `argmax()` 函数 :

`7`

展开数组 :

`[30 40 70 80 20 10 50 90 60]`

沿轴 0 的最大值索引 :

`[1 2 0]`

沿轴 1 的最大值索引 :

`[2 0 1]`

调用 `argmin()` 函数 :

`5`

展开数组中的最小值 :

`10`

沿轴 0 的最小值索引 :

`[0 1 1]`

沿轴 1 的最小值索引 :

`[0 2 0]`

Numpy排序, 条件刷选函数(6/8)

- `numpy.nonzero()` 函数返回输入数组中非零元素的索引。

```
import numpy as np
a = np.array([[30, 40, 0], [0, 20, 10], [50, 0, 60]])
print('我们的数组是 :')
print(a)
print('调用 nonzero() 函数 :')
print(np.nonzero(a))
```



我们的数组是 :

[[30 40 0]

[0 20 10]

[50 0 60]]

调用 nonzero() 函数 :

(array([0, 0, 1, 1, 2, 2], dtype=int64),

array([0, 1, 1, 2, 0, 2], dtype=int64))

Numpy排序, 条件刷选函数(7/8)

- `numpy.where()` 函数返回输入数组中满足给定条件的元素的索引。

```
import numpy as np
x = np.arange(9.).reshape(3, 3)
print('我们的数组是 : ')
print(x)
print('大于 3 的元素的索引 : ')
y = np.where(x > 3)
print(y)
print('使用这些索引来获取满足条件的元素 : ')
print(x[y])
```

输出结果

我们的数组是 :

[[0. 1. 2.]

[3. 4. 5.]

[6. 7. 8.]]

大于 3 的元素的索引 :

(array([1, 1, 2, 2, 2], dtype=int64), array([1, 2, 0, 1, 2],
dtype=int64))

使用这些索引来获取满足条件的元素 :

[4. 5. 6. 7. 8.]

Numpy排序, 条件刷选函数(8/8)

- `numpy.extract()` 函数根据某个条件从数组中抽取元素, 返回满条件的元素。

```
import numpy as np
x = np.arange(9.).reshape(3, 3)
print('我们的数组是 : ')
print(x)
# 定义条件, 选择偶数元素
condition = np.mod(x, 2) == 0
print('按元素的条件值 : ')
print(condition)
print('使用条件提取元素 : ')
print(np.extract(condition, x))
```

输出结果

我们的数组是 :

[[0. 1. 2.]

[3. 4. 5.]

[6. 7. 8.]]

按元素的条件值 :

[[True False True]

[False True False]

[True False True]]

使用条件提取元素 :

[0. 2. 4. 6. 8.]

Numpy字节交换, 副本和视图(1/8)

- 在几乎所有的机器上, 多字节对象都被存储为连续的字节序列。字节顺序, 是跨越多字节的程序对象的存储规则。
 - 大端模式
 - 指数据的高字节保存在内存的低地址中, 而数据的低字节保存在内存的高地址中, 这样的存储模式有点儿类似于把数据当作字符串顺序处理: 地址由小向大增加, 而数据从高位往低位放; 这和我们的阅读习惯一致。
 - 小端模式
 - 指数据的高字节保存在内存的高地址中, 而数据的低字节保存在内存的低地址中, 这种存储模式将地址的高低和数据位权有效地结合起来, 高地址部分权值高, 低地址部分权值低。
- 例如在 C 语言中, 一个类型为 int 的变量 x 地址为 0x100, 那么其对应地址表达式 &x 的值为 0x100。且 x 的四个字节将被存储在存储器的 0x100, 0x101, 0x102, 0x103 位置。

Numpy字节交换, 副本和视图(2/8)

- `numpy.ndarray.byteswap()`函数将ndarray中每个元素中的字节进行大小端转换。

```
import numpy as np
a = np.array([3, 256, 8755], dtype=np.int16)
print('我们的数组是 :')
print(a)
print('以十六进制表示内存中的数据 :')
print(map(hex, a))
# byteswap() 函数通过传入 true 来原地交换
print('调用 byteswap() 函数 :')
print(a.byteswap(True))
print('十六进制形式 :')
print(map(hex, a))
# 我们可以看到字节已经交换了
```

输出结果

我们的数组是 :

[3 256 8755]

以十六进制表示内存中的数据 :

<map object at 0x000001D4A0C9FD48>

调用 byteswap() 函数 :

[768 1 13090]

十六进制形式 :

<map object at 0x000001D4A0C9FD48>

Process finished with exit code 0

Numpy字节交换, 副本和视图(3/8)

- `numpy.ndarray.byteswap()`函数将ndarray中每个元素中的字节进行大小端转换。

```
import numpy as np
a = np.array([3, 256, 8755], dtype=np.int16)
print('我们的数组是 :')
print(a)
print('以十六进制表示内存中的数据 :')
print(map(hex, a))
# byteswap() 函数通过传入 true 来原地交换
print('调用 byteswap() 函数 :')
print(a.byteswap(True))
print('十六进制形式 :')
print(map(hex, a))
# 我们可以看到字节已经交换了
```

输出结果

我们的数组是 :

[3 256 8755]

以十六进制表示内存中的数据 :

<map object at 0x000001D4A0C9FD48>

调用 byteswap() 函数 :

[768 1 13090]

十六进制形式 :

<map object at 0x000001D4A0C9FD48>

Process finished with exit code 0

Numpy字节交换, 副本和视图(4/8)

- 副本是一个数据的完整的拷贝, 如果我们对副本进行修改, 它不会影响到原始数据, 物理内存不在同一位置。
- 视图是数据的一个别称或引用, 通过该别称或引用亦便可访问、操作原有数据, 但原有数据不会产生拷贝。如果我们对视图进行修改, 它会影响原始数据, 物理内存存在同一位置。

Numpy字节交换, 副本和视图(5/8)

- 副本是一简单的赋值不会创建数组对象的副本。相反, 它使用原始数组的相同id()来访问它。 id()返回Python对象的通用标识符, 类似于 C 中的指针。
- 此外, 一个数组的任何变化都反映在另一个数组上。例如, 一个数组的形状改变也会改变另一个数组的形状。

```
import numpy as np
a = np.arange(6)
print('我们的数组是 :')
print(a)
print('调用 id() 函数 :')
print(id(a))
print('a 赋值给 b :')
b = a
print(b)
print('b 拥有相同 id() :')
print(id(b))
print('修改 b 的形状 :')
b.shape = 3, 2
print(b)
print('a 的形状也修改了 :')
print(a)
```



我们的数组是 :
[0 1 2 3 4 5]
调用 id() 函数 :
2027100543952
a 赋值给 b :
[0 1 2 3 4 5]
b 拥有相同 id() :
2027100543952
修改 b 的形状 :
[[0 1]
 [2 3]
 [4 5]]
a 的形状也修改了 :
[[0 1]
 [2 3]
 [4 5]]

Numpy字节交换, 副本和视图(6/8)

- `ndarray.view()` 方会创建一个新的数组对象, 该方法创建的新数组的维数变化不会改变原始数据的维数。

```
import numpy as np
# 最开始 a 是个 3X2 的数组
a = np.arange(6).reshape(3, 2)
print('数组 a : ')
print(a)
print('创建 a 的视图 : ')
b = a.view()
print(b)
print('两个数组的 id() 不同 : ')
print('a 的 id() : ')
print(id(a))
print('b 的 id() : ')
print(id(b))
# 修改 b 的形状, 并不会修改 a
b.shape = 2, 3
print('b 的形状 : ')
print(b)
print('a 的形状 : ')
print(a)
```

输出结果

数组 a :

```
[[0 1]
 [2 3]
 [4 5]]
```

创建 a 的视图 :

```
[[0 1]
 [2 3]
 [4 5]]
```

两个数组的 id() 不同 :

a 的 id() :

2284162883088

b 的 id() :

2284203620656

b 的形状 :

```
[[0 1 2]
 [3 4 5]]
```

a 的形状 :

```
[[0 1]
 [2 3]
 [4 5]]
```

Numpy字节交换, 副本和视图(7/8)

- 使用切片创建视图修改数据会影响到原始数组 :

```
import numpy as np
arr = np.arange(12)
print('我们的数组 :')
print(arr)
print('创建切片 :')
a = arr[3:]
b = arr[3:]
a[1] = 123
b[2] = 234
print(arr)
print(id(a), id(b), id(arr[3:]))
```



我们的数组 :

[0 1 2 3 4 5 6 7 8 9 10 11]

创建切片 :

[0 1 2 3 123 234 6 7 8 9 10 11]
2978709195248 2978709194960
2978709195344

Numpy字节交换, 副本和视图(8/8)

- `ndarray.copy()` 函数创建一个副本。对副本数据进行修改, 不会影响到原始数据, 它们物理内存不在同一位置。

```
import numpy as np
a = np.array([[10, 10], [2, 3], [4, 5]])
print('数组 a : ')
print(a)
print('创建 a 的深层副本 : ')
b = a.copy()
print('数组 b : ')
print(b)
# b 与 a 不共享任何内容
print('我们能够写入 b 来写入 a 吗?')
print(b is a)
print('修改 b 的内容 : ')
b[0, 0] = 100
print('修改后的数组 b : ')
print(b)
print('a 保持不变 : ')
print(a)
```



数组 a :
[[10 10]
[2 3]
[4 5]]
创建 a 的深层副本 :
数组 b :
[[10 10]
[2 3]
[4 5]]
我们能够写入 b 来写入 a 吗?
False
修改 b 的内容 :
修改后的数组 b :
[[100 10]
[2 3]
[4 5]]
a 保持不变 :
[[10 10]
[2 3]
[4 5]]

注意 : `numpy.ndarray` 与 `list` 中深拷贝和浅拷贝的区别

矩阵库与线性代数(1/13)

- NumPy中除了可以使用`numpy.transpose`函数来对换数组的维度, 还可以使用 `T` 属性。

```
import numpy as np
a = np.arange(12).reshape(3, 4)
print('原数组 :')
print(a)
print('转置数组 :')
print(a.T)
```



原数组 :

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

转置数组 :

```
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

矩阵库与线性代数(2/13)

■ `matlib.empty()`函数返回一个新的矩阵

```
import numpy.matlib
import numpy as np
print(np.matlib.empty((2, 2)))
# 填充为随机数据
```



```
[[7.43982552e+006 3.38226911e-061]
 [1.01820900e+277 1.90886933e+136]]
```

■ `numpy.matlib.zeros()`函数创建一个以0填充的矩阵

```
import numpy.matlib
import numpy as np
print(np.matlib.zeros((2, 2)))
```



```
[[0. 0.]
 [0. 0.]]
```

■ `numpy.matlib.ones()`函数创建一个以 1 填充的矩阵

```
import numpy.matlib
import numpy as np
print(np.matlib.ones((2, 2)))
```



```
[[1. 1.]
 [1. 1.]]
```

矩阵库与线性代数(3/13)

- `numpy.matlib.eye()` 函数返回一个矩阵, 对角线元素为 1, 其他位置为零

```
import numpy.matlib
import numpy as np
print(np.matlib.eye(n=3, M=4, k=0, dtype=float))
```

输出结果

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
```

- `numpy.matlib.identity()` 函数返回给定大小的单位矩阵

```
import numpy.matlib
import numpy as np
# 大小为 5, 类型位浮点型
print(np.matlib.identity(5, dtype=float))
```

输出结果

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```


矩阵库与线性代数(4/13)

- `numpy.matlib.rand()`函数创建一个给定大小的矩阵, 数据是随机填充的。

```
import numpy.matlib
import numpy as np
print(np.matlib.rand(3, 3))
```

输出结果

```
[[0.05952765 0.80925609 0.7346986 ]
 [0.08035761 0.30378389 0.04821102]
 [0.37576726 0.6465993  0.32469582]]
```

- 矩阵总是二维的, 而`ndarray`是一个n维数组。两个对象都是可互换的。

```
import numpy as np
i = np.matrix('1,2;3,4')
print(i)
j = np.asarray(i)
print(j)
k = np.asmatrix(j)
print(k)
```

输出结果

```
[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
[[1 2]
 [3 4]]
```

矩阵库与线性代数(5/13)

- NumPy 提供了线性代数函数库 linalg, 该库包含了线性代数所需的所有功能

函数	描述
dot	两个数组的点积, 即元素对应相乘。
vdot	两个向量的点积
inner	两个数组的内积
matmul	两个数组的矩阵积
determinant	数组的行列式
solve	求解线性矩阵方程
inv	计算矩阵的乘法逆矩阵

矩阵库与线性代数(6/13)

- `numpy.dot()`对于两个一维的数组, 计算的是这两个数组对应下标元素的乘积和(数学上称之为向量点积); 对于二维数组, 计算的是两个数组的矩阵乘积; 对于多维数组, 它的通用计算公式如下, 即结果数组中的每个元素都是: 数组a的最后一维上的所有元素与数组b的倒数第二位上的所有元素的乘积和:
 - $\text{dot}(a, b)[i,j,k,m] = \text{sum}(a[i,j,:] * b[k,:,m])$

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[11, 12], [13, 14]])
print(np.dot(a, b))
```



```
[[37 40]
 [85 92]]
```

计算式为:

$[[1*11+2*13, 1*12+2*14], [3*11+4*13, 3*12+4*14]]$

矩阵库与线性代数(7/13)

- `numpy.vdot()`函数是两个向量的点积。如果第一个参数是复数，那么它的共轭复数会用于计算。如果参数是多维数组，它会被展开。

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[11, 12], [13, 14]])
# vdot 将数组展开计算内积
print(np.vdot(a, b))
```



130

计算式为：

$$1*11 + 2*12 + 3*13 + 4*14 = 130$$

矩阵库与线性代数(8/13)

- `numpy.inner()`函数返回一维数组的向量内积。对于更高的维度，它返回最后一个轴上的和的乘积。

```
import numpy as np
print(np.inner(np.array([[1, 2, 3],[4,5,6]]), np.array([[0, 1, 0],[1,0,0]])))
# 等价于  $1*0+2*1+3*0$ 
```



```
[[2 1]
 [5 4]]
```

矩阵库与线性代数(9/13)

- `numpy.inner()`函数返回一维数组的向量内积。对于更高的维度，它返回最后一个轴上的和的乘积。

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print('数组 a : ')
print(a)
b = np.array([[11, 12], [13, 14]])
print('数组 b : ')
print(b)
print('内积 : ')
print(np.inner(a, b))
```

输出结果

数组 a :

[[1 2]

[3 4]]

数组 b :

[[11 12]

[13 14]]

内积 :

[[35 41]

[81 95]]

矩阵库与线性代数(10/13)

- `numpy.matmul` 函数返回两个数组的矩阵乘积。

```
import numpy as np  
a = [[1, 0], [0, 1]]  
b = [[4, 1], [2, 2]]  
print(np.matmul(a, b))
```



```
[[4 1]  
 [2 2]]
```

- 二维和一维运算

```
import numpy as np  
a = [[1, 2], [3, 1]]  
b = [1, 2]  
print(np.matmul(a, b))  
print(np.matmul(b, a))
```



```
[5 5]  
[7 4]]
```

矩阵库与线性代数(11/13)

- `numpy.linalg.det()` 函数计算输入矩阵的行列式。

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print(np.linalg.det(a))
```

输出结果

-2.

```
import numpy as np
b = np.array([[6, 1, 1], [4, -2, 5], [2, 8, 7]])
print(b)
print(np.linalg.det(b))
print(6 * (-2 * 7 - 5 * 8) - 1 * (4 * 7 - 5 * 2) + 1 * (4 * 8 - -2 * 2))
```

输出结果

```
[[ 6  1  1]
 [ 4 -2  5]
 [ 2  8  7]]
-306.0
-306
```


矩阵库与线性代数(12/13)

- `numpy.linalg.solve()`函数给出了矩阵形式的线性方程的解。

```
import numpy as np
A=np.array([[1,1,1],[0,2,5],[2,5,-1]])
B=np.array([6,-4,27])
print(np.linalg.solve(A,B))
```



[5. 3. -2.]

- `numpy.linalg.inv()` 函数计算矩阵的乘法逆矩阵。

```
import numpy as np
x = np.array([[1, 2], [3, 4]])
y = np.linalg.inv(x)
print(x)
print(y)
print(np.dot(x, y))
```



```
[[1 2]
 [3 4]
 [-2.  1.]
 [ 1.5 -0.5]]
[[1.00000000e+00  1.11022302e-16]
 [0.00000000e+00  1.00000000e+00]]
```

矩阵库与线性代数(13/13)

■ 线性方程计算

```
import numpy as np
a = np.array([[1, 1, 1], [0, 2, 5], [2, 5, -1]])
print('数组 a : ')
print(a)
ainv = np.linalg.inv(a)
print('a 的逆 : ')
print(ainv)
print('矩阵 b : ')
b = np.array([[6], [-4], [27]])
print(b)
print('计算 : A(-1)B : ')
x = np.linalg.solve(a, b)
print(x)
# 这就是线性方向 x = 5, y = 3, z = -2
的解
```

输出结果

数组 a :

```
[[ 1  1  1]
 [ 0  2  5]
 [ 2  5 -1]]
```

a 的逆 :

```
[[ 1.28571429 -0.28571429 -0.14285714]
 [-0.47619048  0.14285714  0.23809524]
 [ 0.19047619  0.14285714 -0.0952381 ]]
```

矩阵 b :

```
[[ 6]
 [-4]
 [27]]
```

计算 : A⁽⁻¹⁾B :

```
[[ 5.]
 [ 3.]
 [-2.]]
```

Numpy IO与Matplotlib(1/13)

- `numpy.save()` 函数将数组保存到以 `.npy` 为扩展名的文件中。

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
# 保存到 outfile.npy 文件上
np.save('outfile.npy', a)
# 保存到 outfile2.npy 文件上, 如果文件路径末尾没有扩展名
.npy, 该扩展名会被自动加上
np.save('outfile2', a)
```

Numpy IO与Matplotlib(2/13)

■ load()函数来读取数据

```
import numpy as np
b = np.load('outfile.npy')
print(b)
```

输出结果 → [1 2 3 4 5]

■ numpy.savez() 函数将多个数组保存到以 npz 为扩展名的文件中。

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.arange(0, 1.0, 0.1)
c = np.sin(b)
np.savez("runoob.npz", a, b, c)
r = np.load("runoob.npz")
print(r.files) # 查看各个数组名称
print(r["arr_0"]) # 数组 a
print(r["arr_1"]) # 数组 b
print(r["arr_2"]) # 数组 c
```

输出结果 →

```
['arr_0', 'arr_1', 'arr_2']
[[1 2 3]
 [4 5 6]]
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
[0.          0.09983342 0.19866933
 0.29552021 0.38941834 0.47942554
 0.56464247 0.64421769 0.71735609
 0.78332691]
```

Numpy IO与Matplotlib(3/13)

- **savetxt()** 函数是以简单的文本文件格式存储数据，对应的使用 **loadtxt()** 函数来获取数据。

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
np.savetxt('out.txt', a)
b = np.loadtxt('out.txt')
print(b)
```



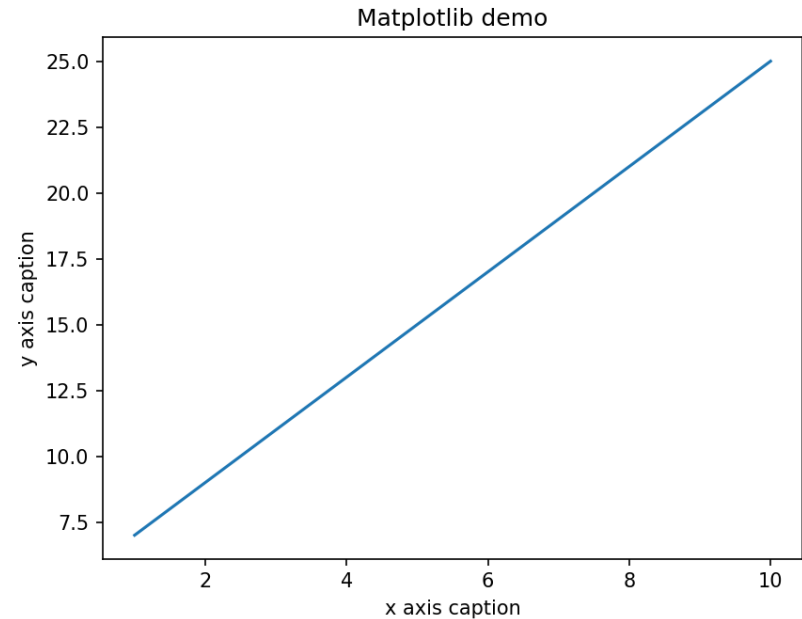
[1. 2. 3. 4. 5.]

Numpy IO与Matplotlib(4/13)

■ Matplotlib 是 Python 的绘图库

```
import numpy as np
from matplotlib import pyplot as plt
x = np.arange(1, 11)
y = 2 * x + 5
plt.title("Matplotlib demo")
plt.xlabel("x axis caption")
plt.ylabel("y axis caption")
plt.plot(x, y)
plt.show()
```

输出结果



Thank you!